

> Menu

Getting Started & Local Development

▼ Relevant source files

serve.js

This page details how to set up and run the EULA project locally using the provided Node.js server, `serve.js`. It covers the server's functionality, including port configuration, MIME type handling, and a crucial security measure against path traversal. Understanding local development is also key to comprehending why running on `localhost` is essential for Progressive Web App (PWA) features like installation.

Running the Local Server

The project includes a simple Node.js HTTP server, `serve.js`, designed to serve the static files over `localhost`. This is important because PWA features, such as the install prompt, require a secure context, which `localhost` provides by default `serve.js` 1-2

To start the server, navigate to the project's root directory in your terminal and execute the following command:

```
node serve.js
```

Upon successful execution, the server will log the address where the application is being served

```
serve.js 32 :
```

```
Eternal Terms serving at http://localhost:8124/
```

Ask Devin about dev-justin-time/EULA

You can then access the application by navigating to `http://localhost:8124/` in your web browser.

⚡ Fast Configuration



The `serve.js` script is configured with the following parameters:

- **Root Directory:** The server's root directory is set to the directory where `serve.js` resides, using `__dirname` `serve.js` 7 This ensures that only files within the project directory are served.
- **Port:** The server listens on port `8124` by default. This can be overridden by setting the `PORT` environment variable `serve.js` 8 For example: `PORT=3000 node serve.js`.

MIME Type Handling

The server includes a `MIME` object that maps common file extensions to their corresponding MIME types `serve.js` 10-20 This is crucial for browsers to correctly interpret and render the served content.

Extension	MIME Type
<code>.html</code>	<code>text/html</code>
<code>.css</code>	<code>text/css</code>
<code>.js</code>	<code>text/javascript</code>
<code>.mp3</code>	<code>audio/mpeg</code>
<code>.png</code>	<code>image/png</code>
<code>.ico</code>	<code>image/x-icon</code>
<code>.webmanifest</code>	<code>application/manifest+json</code>
<code>.json</code>	<code>application/json</code>
<code>.svg</code>	<code>image/svg+xml</code>

If a requested file's extension is not found in the `MIME` map, the server defaults to `application/octet-stream` `serve.js` 29

Request Handling and Path Traversal Guard

The `http.createServer` callback handles incoming requests `serve.js` 22

Ask Devin about dev-justin-time/EULA

1. **Path Normalization:** The requested URL path (`req.url`) is decoded and normalized. If the path is `/` , it defaults to `/index.html` `serve.js` 23-24

2. **File Resolution:** The `path.normalize` and `path.join` functions are used to construct the full file path, ensuring it remains within the `ROOT` directory `serve.js` 25
3. **Path Traversal Guard:** A critical security measure is implemented to prevent directory traversal attacks. The `if (!file.startsWith(ROOT))` check ensures that the resolved file path does not escape the designated `ROOT` directory `serve.js` 26 If an attempt to access a file outside the root is detected, the server responds with a `403 Forbidden` status `serve.js` 26
4. **File Serving:** If the file is valid and within the root, `fs.readFile` attempts to read the file `serve.js` 27
 - On success, the server responds with a `200 OK` status, setting the `Content-Type` header based on the `MIME` map, and sends the file data `serve.js` 29-30
 - On error (e.g., file not found), the server responds with a `404 Not Found` status `serve.js` 28

The following diagram illustrates the request handling flow within `serve.js` :

Sources: `serve.js` 22-31

`serve.js` System Overview

DeepWiki dev-justin-time/EULA

 Edit Wiki

 Share



Secure Context and PWA Installation

Running the application on `localhost` is crucial for enabling Progressive Web App (PWA) features. Modern web browsers consider `localhost` a secure context, even without HTTPS `serve.js` 1-2

This allows features like Service Workers and the Web App Manifest to function correctly, which are prerequisites for the "Add to Home Screen" or "Install App" prompt to appear. Without a secure context, the browser will not offer to install the PWA, limiting its offline capabilities and native-like

Ask Devin about dev-justin-time/EULA

Sources: `serve.js` 1-2

Ask Devin about dev-justin-time/EULA

